A Project Report

on

"Learning Management system with smart study planner"

AKS UNIVERSITY, SATNA

B.Tech,CSE (AI & DS),6th Semester

Submitted by

Suryansh Pratap Singh (B2355R10195108)

Program : M.P. ONLINE

Department of Computer Science & Engineering

AKS University, Satna (M.P.)

CERTIFICATE

This is to certify that the project report entitled **"**Learning Management System with Smart Study Planner**" submitted by "**Suryansh Pratap Singh**" in partial fulfillment of the requirements for the award of the degree of "**Bachelor of Technology in Computer Science and Engineering**" at **AKS University, Satna**, is a bona fide record of project work carried out under the guidance and supervision of the undersigned.

The work presented in this report is original and has not been submitted, either in part or in full, for the award of any other degree or diploma to any other institution or university.

All assistance and sources of information used during the development of the project have been duly acknowledged in the report.

Place: AKSU, Satna
Date: ___ / ___ / _____

External Signature: _____
Guide Signature: _____
Head of Department: _____
Dean: _____

SELF DECLARATION

I hereby declare that the work presented in this project report entitled "Learning Management System with Smart Study Planner" is an authentic record of my own work carried out in partial fulfillment of the requirements for the award of the degree of “B.Tech” in the Department of Computer Science and Engineering, AKS University, Satna (M.P.).

I further declare that this work has not been submitted earlier for the award of any degree or diploma to any other institute or university.

Signature of Candidate: _____

Name: Suryansh Pratap Singh

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, faculty members, and the Department of Computer Science and Engineering, AKS University, Satna, for their guidance, support, and encouragement throughout the development of this project.

I am especially thankful to my guide for providing valuable suggestions during the planning, design, implementation, and evaluation stages of the project. Their academic support helped shape the direction of this work and improve the quality of the final system.

I would also like to thank my parents and friends for their continuous moral support, motivation, and encouragement. Their confidence in me helped me stay focused and complete the project successfully.

ABSTRACT

The “ Learning Management System with Smart Study Planner ”is a full-stack web application designed to help students organize courses, manage academic tasks, schedule study sessions, and monitor learning progress in a centralized digital platform. Traditional study management methods often rely on notebooks, disconnected apps, or manual reminders, which can make it difficult for students to track deadlines, distribute workload, and understand their progress over time.

This project addresses that problem by integrating user authentication, course management, task tracking, automated study planning, and analytics into one system. The application is built using “React with Vite” for the frontend, “FastAPI” for the backend, “PostgreSQL”for data storage, and “SQLAlchemy” for database interaction. JWT-based authentication is used to provide secure user access, while the dashboard and analytics modules present progress in a simple visual format.

The core strength of the project is its “smart study planner”, which automatically generates study sessions for pending tasks based on deadlines, estimated hours, and task priority. This helps students use their available time effectively and reduce the risk of missed deadlines.

The system is modular, scalable, and suitable for academic use. It demonstrates the practical integration of modern web technologies to solve a real-world student productivity problem through automation, structured planning, and data-driven progress tracking.

Keywords: Learning Management System, Study Planner, React, FastAPI, PostgreSQL, JWT Authentication, Task Scheduling, Analytics.

TABLE OF CONTENTS

1. Introduction ……. 6-7
2. Objective of The Project …….8-9
3. Scope of The Project …… .10-12
4. Problem Definition …… .13-14
5. Benefits of The Project…….15-16
6. Project Modules…….17-20
7. Technical Overview……21-26
8. System Configuration …… 27-30
9.Project Explanation …….31-32
10. Output Screens…….. 33-34
11. Result and Analysis……...35-37
12. software testing …….38-40
13. Conclusion…….41-42
14.References……43

Introduction

The rapid growth of digital education, self-paced learning, and academic workload management has created a strong demand for systems that can help students manage their study process more efficiently. In many colleges and universities, students are expected to track multiple subjects, assignments, deadlines, and revision plans simultaneously. However, most learners still depend on scattered tools such as notebooks, mobile reminders, spreadsheets, or messaging apps to organize their academic work. These methods are often inconsistent, difficult to maintain, and not connected to actual progress data.

A Learning Management System (LMS) provides a structured digital environment where students can organize their learning activities in one place. In its modern form, an LMS is no longer limited to course material delivery. It can also help students plan work, manage deadlines, measure progress, and make better decisions about how to use their time. When such a system is enhanced with a study planner and analytics, it becomes a productivity assistant rather than just a storage platform.

The present project, **Learning Management System with Smart Study Planner**, has been developed to address this need. It allows users to create an account, log in securely, add courses, create tasks associated with courses, define deadlines and estimated hours, and generate study sessions automatically. The system also includes dashboards and analytics views so that the learner can see course-wise progress, total study hours, completed tasks, pending tasks, and upcoming deadlines.

This project combines the strengths of modern frontend and backend technologies. The frontend is developed using **React** and **Tailwind CSS**, which provide an interactive and responsive user interface. The backend is built using **FastAPI**, a lightweight and efficient Python web framework suited for REST API development. **PostgreSQL** is used as the relational database, while **SQLAlchemy** handles data modeling and persistence. JWT-based authentication ensures secure user access and protected routes.

One of the most useful features of this project is the **smart planner module**, which generates a study plan automatically for pending tasks by considering deadlines, priority, and estimated workload. This makes the system more intelligent and useful in real academic scenarios, where students often know what they have to do but struggle to decide when and how to do it.

The project is designed for simplicity, modularity, and practical value. It can be used by college students, coaching learners, and self-learners who need a centralized system for managing coursework and study schedules. It also has enough modular structure to be extended in the future with features such as notifications, teacher dashboards, file uploads, calendar sync, or AI-based recommendations.

In summary, this LMS project is a practical attempt to combine authentication, course management, task planning, time scheduling, and analytics into one coherent academic productivity platform. It demonstrates how full-stack development can be applied to solve a meaningful real-world problem for students.

Objective of The Project

The primary objective of the " Learning Management System with Smart Study Planner " is to design and develop a modern, web-based academic management platform that helps students organize their learning activities in a systematic, secure, and efficient manner. The system aims to reduce the difficulties students face while handling multiple courses, assignments, deadlines, revision schedules, and study targets at the same time. Instead of depending on scattered tools such as notebooks, reminders, and separate applications, the project provides a unified environment where academic information can be created, managed, and monitored in one place.

Another major objective of the project is to improve **time management and planning efficiency**. In many cases, students know what work is pending but struggle to decide when to start, how to prioritize tasks, and how to distribute effort across the week. This project addresses that issue by providing a smart study planner that can automatically generate study sessions based on task deadlines, priority, and estimated workload. The purpose is not only to store academic information, but also to convert that information into a meaningful and actionable study schedule.

The project also aims to introduce **data-driven learning support**. Through dashboard and analytics modules, students can observe their performance visually in terms of completed tasks, pending work, study hours, course progress, and upcoming deadlines. This helps users understand their own learning patterns, stay motivated, and make better academic decisions.

From a technical perspective, the objective is to implement a complete **full-stack software solution** using modern technologies such as React, FastAPI, PostgreSQL, and SQLAlchemy. The project demonstrates how frontend, backend, authentication, database design, scheduling logic, and data visualization can be integrated into one real-world application. It also aims to create a modular architecture so that future features such as notifications, faculty roles, content sharing, report generation, and AI-based recommendations can be added without redesigning the entire system.

The specific objectives of the project are as follows:

- To design a secure user authentication system using signup, login, token generation, and protected route access.

- To create a centralized academic platform where students can manage multiple courses from a single interface.

- To allow users to create, update, view, and delete tasks associated with individual courses.

- To include task properties such as description, deadline, estimated study hours, and priority so that academic work can be tracked in detail.

- To develop a smart planning mechanism that automatically creates study sessions for pending tasks.

- To provide a manual planning option for users who want to schedule their study sessions directly.

- To build a dashboard that gives an overall summary of student activity and academic workload.

- To provide analytics in graphical form so that progress and study habits can be understood more clearly.

- To reduce the chances of missed deadlines by highlighting urgent tasks and upcoming academic commitments.

- To improve self-discipline and productivity by helping students follow a structured study plan.

- To provide a simple, responsive, and user-friendly interface that can be used easily by students with basic computer knowledge.

- To implement a scalable project structure that supports future improvements and additional modules.

- To demonstrate practical application of full-stack development concepts in an educational domain.

Scope of The Project

The scope of the **" Learning Management System with Smart Study Planner "** includes the design and implementation of a complete student-focused academic management platform that supports planning, organization, monitoring, and basic performance analysis. The project is intended to solve the everyday academic difficulties faced by students who need a better way to manage courses, assignments, deadlines, and study time in a single digital environment.

This project primarily focuses on the ****student-side learning workflow****. It begins from user registration and secure login, extends to course and task management, and reaches up to study planning and visual analytics. In this way, the scope is not limited to simple data entry or record keeping. Instead, it includes intelligent assistance through automated planning and progress-oriented dashboards, making the application useful for both academic organization and self-management.

The current scope of the project includes the following functional areas:

- User registration, login, logout, and secure access to personalized data
- Creation and management of multiple academic courses
- Task creation under each course with title, description, deadline, estimated study hours, and priority
- Viewing, updating, deleting, and filtering tasks according to course and completion status
- Marking tasks as completed for better progress tracking
- Creating manual study sessions for specific dates and time slots
- Automatically generating study schedules based on task urgency, priority, and workload
- Displaying dashboard summaries such as total courses, completed tasks, pending tasks, and study hours
- Showing upcoming deadlines and course-wise progress
- Presenting analytical information through charts and summary reports

The scope also includes the **technical development** of a full-stack application using modern web technologies. On the frontend side, the project covers responsive interface design, route protection, and user interaction. On the backend side, it includes API development, schema validation, authentication handling, database modeling, and business logic for study scheduling and analytics. Therefore, the project has both a functional scope and a technical scope.

From a practical point of view, the system can be used by:

- College students managing semester subjects and assignments
- Coaching students tracking study tasks and revision plans
- Self-learners organizing courses, practice work, and deadlines
- Small academic groups who need a simple personal productivity platform

Although the present version is centered on an individual learner, the system has been designed with enough flexibility to support future enhancements. The architecture can be extended to include:

- Faculty or administrator roles
- Learning content upload and sharing
- Notices and reminders
- Attendance tracking
- Assignment submissions
- Exam scheduling and result management
- Email or SMS notifications
- AI-based study recommendations
- Calendar synchronization and report export features

However, these advanced features are outside the current implementation scope and may be considered in future versions of the project.

It is also important to define the **limitations of the present scope**. The current system does not provide live classroom integration, video lectures, online examinations, student-teacher communication modules, or institution-wide management controls. It is not intended to replace a full enterprise LMS at this stage. Instead, it is designed as a focused academic planning and productivity tool with intelligent scheduling support.

Thus, the scope of the project is broad enough to cover meaningful academic management features, yet sufficiently focused to ensure proper implementation, testing, and usability. It addresses the immediate needs of students while establishing a strong foundation for future growth into a more comprehensive educational platform.

Problem Definition

In the present academic environment, students are expected to manage a wide range of learning responsibilities simultaneously. These may include attending classes, preparing notes, completing assignments, revising topics, working on projects, and preparing for internal or external examinations. As the number of academic responsibilities increases, students often find it difficult to organize their work effectively. The absence of a structured planning system creates confusion, poor prioritization, irregular study habits, and a high risk of missed deadlines.

One of the most common problems faced by students is the **fragmentation of academic information**. Course details, assignment deadlines, revision plans, and study schedules are often maintained in different places such as notebooks, phone reminders, spreadsheets, messaging applications, or memory. Since these tools are not integrated, students may lose track of important work, repeat efforts unnecessarily, or fail to maintain consistency in their study process.

Another major problem is the **lack of effective prioritization**. Students usually know what tasks they need to complete, but they often do not know which task should be done first or how much time should be assigned to each one. As a result, important academic work may be postponed, less important activities may consume more time, and deadlines may approach without proper preparation. This issue becomes even more serious during examination periods or project submission weeks.

The problem is not limited to storing task lists. A deeper issue is the **absence of intelligent planning support**. Most students are left to manually decide how to convert pending assignments and learning goals into an actual day-by-day study routine. Manual planning is often inconsistent, unrealistic, or ignored after a short period. Without a system that can automatically organize tasks according to urgency, workload, and available time, students may struggle to maintain discipline and balance in their academic schedule.

There is also a clear problem in terms of **progress visibility and self-monitoring**. In traditional study methods, students may not have a clear picture of how much work they have completed, how many tasks are still pending, how many study hours have been invested, or

which courses require more attention. Without measurable feedback, it becomes difficult to evaluate personal performance or make improvements in study habits.

The major problems identified in the existing situation are as follows:

- No centralized platform to manage courses, tasks, deadlines, and study sessions together
- Dependence on scattered manual tools for academic planning
- Difficulty in prioritizing tasks according to urgency, deadline, and importance
- Lack of an automated mechanism to generate an effective study schedule
- Poor visibility into pending work, completed work, and course-wise progress
- Higher chances of missing deadlines or creating an unbalanced study routine
- Extra mental burden caused by repeatedly deciding what to study next
- Limited analytical insight into learning behavior and productivity
- Inability of traditional methods to scale when workload increases

These problems directly affect a student's efficiency, consistency, and academic performance. When planning is weak, even capable students may underperform because of poor workload distribution and lack of structured tracking. Therefore, the real problem is not simply the absence of a digital task list, but the absence of an integrated academic productivity system that combines organization, planning, and analysis in one place.

To address this challenge, there is a need for a system that can securely store user-specific academic data, organize courses and tasks, assist in scheduling study sessions, and provide meaningful insights into progress and deadlines. The ****Learning Management System with Smart Study Planner**** is proposed as a solution to this problem. It aims to bridge the gap between task management and study planning by providing a single platform where students can not only record their academic responsibilities, but also manage them intelligently and effectively.

Benefits of The Project

The **" Learning Management System with Smart Study Planner "** provides several important benefits to students by combining academic organization, time planning, and progress monitoring into one integrated platform. The project is designed not only to store information, but also to help users make better academic decisions and improve the way they manage their day-to-day study activities.

One of the biggest benefits of the system is ****centralized academic management****. Instead of using multiple disconnected tools such as notebooks, reminder applications, spreadsheets, and chat messages, the student can maintain courses, tasks, deadlines, and study schedules in a single system. This reduces confusion and makes academic information easier to access, update, and monitor.

Another major benefit is ****better time management****. Many students face difficulty in deciding when to study and how to divide their available time among different subjects. The smart planner feature helps solve this problem by automatically generating study sessions from pending tasks. This makes planning faster, more practical, and more consistent than manual methods.

The system also provides ****improved task prioritization****. Since each task can include deadline, priority level, and estimated hours, the student can clearly identify what work is urgent and what can be scheduled later. This helps reduce procrastination and improves the overall quality of academic planning.

An additional benefit is ****higher visibility of academic progress****. Through dashboards and analytics, students can understand how much work has been completed, how many tasks are still pending, how much time has been spent studying, and which courses need more attention. This kind of feedback is useful for self-improvement, motivation, and regular performance tracking.

The major benefits of the project are as follows:

- It centralizes all course, task, and study planning information in one digital platform.

- It reduces dependence on scattered manual tools and unstructured reminders.

- It improves academic organization by linking tasks directly to courses.

- It helps students manage study time more effectively through scheduled sessions.
- It reduces the mental effort required to decide what to study next.
- It improves deadline awareness through planner-based scheduling and upcoming deadline tracking.
- It enables better prioritization of work based on urgency, importance, and estimated effort.
- It supports regular monitoring of progress through summary cards, charts, and analytics.
- It encourages consistency, discipline, and better study habits.
- It helps reduce missed assignments and last-minute workload pressure.
- It provides secure user-specific access through authentication and protected routes.
- It offers a simple and user-friendly interface suitable for student use.
- It can be expanded in the future for faculty, classroom, or institutional use.

From a broader perspective, the project also has **technical and developmental benefits**. It demonstrates the practical use of full-stack development concepts such as frontend-backend integration, REST API communication, authentication, database design, and data visualization. As a result, the project is beneficial not only to the end user, but also as a strong academic software engineering case study.

Therefore, the system is especially useful for learners who need structure, visibility, automation, and better decision-making support in their academic workflow. By improving planning, monitoring, and organization, the project contributes directly to better productivity and more effective learning management.

Project Modules

The proposed system is divided into several functional modules so that each part of the application can handle a specific responsibility in an organized and manageable way. This modular design improves maintainability, readability, testing, and future scalability of the project. Each module performs an independent role, but all of them work together to provide a smooth academic management experience for the user.

The modules of the project are described below:

1.Authentication Module

The Authentication Module is responsible for handling user identity and securing access to the system. It manages the signup and login process, verifies user credentials, creates secure access tokens, and controls entry to protected pages and API endpoints.

In this project, the authentication system is built using email-based login and JWT token handling. Passwords are stored in encrypted form so that sensitive user information remains protected. Once the user logs in successfully, a token is issued and used to authorize future requests. This ensures that each user can only access their own data such as courses, tasks, study plans, and analytics.

The importance of this module lies in data privacy and user-specific operation. Without authentication, the system could not safely support personalized learning management.

2.Course Management Module

The Course Management Module allows the user to create and manage academic courses in the system. A course acts as the main category under which related tasks and study activities are organized. For example, a student may create separate courses for subjects such as Data Structures, Machine Learning, DBMS, Operating Systems, or Web Development.

This module supports course creation, updating, viewing, and deletion. It allows the user to provide details such as course title, description, and category. By organizing academic data course-wise, the system becomes more structured and easier to use. This module forms the foundation of the overall learning management workflow because all major academic activities are connected to courses.

3. Task Management Module

The Task Management Module is responsible for handling all academic tasks associated with specific courses. A task can represent an assignment, project work item, revision topic, practice session, or any academic goal that the student wants to track.

Each task contains useful details such as title, description, deadline, estimated study hours, priority, and completion status. This module allows users to create tasks, update them, mark them as completed, delete them, and filter them according to course or status. Through this module, students can clearly see what work is pending and what has already been completed.

This module is one of the most important components of the project because it converts general academic goals into clear, trackable, and manageable units of work.

4.Study Planner Module

The Study Planner Module is the core intelligent component of the system. Its main role is to help the student translate pending academic tasks into actual study sessions. This module supports both manual planning and automated planning.

In the manual mode, the user can choose a task, date, and time slot to create a personalized study session. In the automatic mode, the backend scheduling logic analyzes pending tasks, deadlines, priority levels, and estimated hours, then generates a study plan accordingly. This reduces the burden of deciding how to schedule academic work every day.

The planner module improves productivity by making the study process more structured, balanced, and goal-oriented. It is the feature that differentiates this project from an ordinary course-and-task tracker.

5 .Analytics Module

The Analytics Module is designed to provide meaningful insights into the user's academic performance and study behavior. Instead of showing only raw data, this module presents summarized and visual information that helps students evaluate their progress more effectively.

This module includes features such as course-wise progress tracking, total completed tasks, pending tasks, study hours, weekly study patterns, and deadline-based summaries. Visual

representation through charts and graphs makes the information easier to understand and more useful for decision-making.

The purpose of this module is to support self-monitoring and continuous improvement. By viewing progress analytically, students can identify weak areas, improve consistency, and manage workload more intelligently.

6. Dashboard Module

The Dashboard Module acts as the main overview screen of the application. It presents the most important academic information in one place immediately after login. Instead of navigating through multiple sections, the user can quickly understand their current academic status from the dashboard.

The dashboard displays information such as total courses, total tasks, completed tasks, pending tasks, total study hours, progress charts, and upcoming deadlines. It serves as a quick summary of the user's learning activity and provides a convenient starting point for further action.

This module improves usability because it reduces effort and helps the student focus on the most urgent and relevant information at a glance.

7. API Service Module

The API Service Module works as the communication bridge between the frontend and the backend. It is responsible for sending user requests, receiving responses, attaching authentication tokens, and managing the flow of data between the client interface and server-side logic.

In this project, the API layer is implemented using Axios. It handles tasks such as sending login requests, fetching course data, saving tasks, retrieving planner information, and loading analytics. It also ensures that authenticated requests automatically include the required token in the headers.

This module is important because it keeps the frontend code clean and organized while making backend integration more efficient and reusable.

8. Database and Data Management Module

The Database and Data Management Module handles the storage, retrieval, and relationship management of all project data. It maintains user records, course information, task details, study plan entries, and progress-related data in a structured relational format.

The project uses PostgreSQL as the database and SQLAlchemy as the ORM layer. This module ensures that data remains persistent, consistent, and properly connected across different system components. For example, tasks are linked to courses, study plans are linked to tasks, and analytics are generated from stored records.

This module is essential because all other modules depend on it for reliable and secure data operations.

9. Overall Role of Modules

Each module in the system contributes to a specific part of the user experience:

- The Authentication Module secures access.

- The Course Management Module organizes subjects.

- The Task Management Module manages academic work items.

- The Study Planner Module transforms tasks into planned study sessions.

- The Dashboard and Analytics Modules provide visibility and performance tracking.

- The API Service and Database Modules maintain technical integration and data flow.

Together, these modules create a complete and effective academic productivity platform. The modular structure also ensures that future features can be added in a systematic way without disturbing the existing workflow of the application.

Technical Overview

The "Learning Management System with Smart Study Planner"is developed using a modern **client-server full-stack architecture**. In this architecture, the frontend handles user interaction and presentation, the backend manages application logic and request processing, and the database stores all persistent academic data. This separation of concerns makes the system more organized, maintainable, and scalable.

The technical overview of the project can be understood through its major layers and components.

1. Architectural Approach

The application follows a layered web architecture with the following structure:

- **Presentation Layer** for user interface and interaction
- **Application Layer** for request handling and business logic
- **Data Layer** for storing and retrieving structured information

This architecture ensures that each part of the project performs a specific role. The frontend is responsible for displaying pages and collecting user input, the backend is responsible for processing requests and enforcing rules, and the database is responsible for preserving all academic records. Such an approach reduces coupling and improves clarity during development.

2. Frontend Technology Overview

The frontend of the system is built using **React**, which is a component-based JavaScript library for building user interfaces. React is well suited for this project because it supports reusable components, dynamic data updates, and page-based routing with good performance.

The frontend also uses the following technologies:

- **Vite** as the development server and build tool for fast compilation and optimized production builds
- **Tailwind CSS** for responsive and utility-based styling
- **React Router** for navigation between pages such as Login, Signup, Dashboard, Courses, Tasks, Planner, and Analytics
- **Axios** for API communication with the backend
- **Recharts** for graphical visualization of analytics and progress data

The frontend is responsible for:

- Displaying forms for login, signup, courses, tasks, and study sessions
- Managing user interaction and local page state
- Sending requests to backend APIs
- Rendering dashboard summaries and charts
- Protecting pages based on authentication state

Because the UI is component-based, the application remains clean and modular. Pages such as Dashboard, Courses, Tasks, Planner, and Analytics are separated logically, which makes the code easier to manage and extend.

3. Backend Technology Overview

The backend of the project is developed using **FastAPI**, a modern Python web framework designed for building high-performance APIs. FastAPI is appropriate for this project because it supports clean route definitions, automatic data validation, strong typing, and efficient request handling.

The backend uses the following important tools and libraries:

- **FastAPI** for REST API development
- **SQLAlchemy** for ORM-based database interaction

- **Pydantic** for validating request and response data
- **python-jose** for JWT token creation and verification
- **bcrypt** for secure password hashing

The backend is responsible for:

- Managing user authentication and authorization
- Processing CRUD operations for courses and tasks
- Generating and updating study plans
- Calculating analytics and progress summaries
- Enforcing user-specific access to data
- Interacting with the database through ORM models and sessions

The backend is structured into routers, schemas, models, utilities, and configuration files. This structure improves separation of responsibilities and supports future expansion.

4.API Design and Communication

The system follows a **RESTful API communication model**. The frontend sends HTTP requests to the backend, and the backend responds with JSON data. Each functional area of the project is represented through dedicated API routes such as:

- Authentication routes for signup, login, and current-user access
- Course routes for adding, editing, viewing, and deleting courses
- Task routes for task management and completion tracking
- Planner routes for study session creation and auto scheduling
- Analytics routes for summaries, progress data, and weekly study hours

Axios is used on the frontend to simplify API calls. An interceptor automatically attaches the JWT token to authorized requests, which ensures secure communication between frontend and backend without repeating authentication logic in every request.

5. Database Overview

The project uses **PostgreSQL** as the relational database management system. PostgreSQL is a reliable and powerful database suited for structured data storage and relationship handling. Since the project contains multiple connected entities such as users, courses, tasks, and study plans, a relational database is an appropriate choice.

The main database tables are:

- `users`
- `courses`
- `tasks`
- `study_plans`
- `progress`

These tables are connected through foreign key relationships. For example:

- One user can have many courses
- One course can have many tasks
- One task can be scheduled into study plans
- Progress records can be associated with users, courses, and tasks

SQLAlchemy is used as the ORM layer, which allows the backend to interact with database tables using Python classes instead of raw SQL in most situations. This improves code readability and reduces repetitive database handling.

6. Security Overview

Security is an important technical aspect of the project. Since the system stores user-specific academic information, unauthorized access must be prevented.

The project implements the following security mechanisms:

- Password hashing using bcrypt

- JWT-based authentication for secure sessions
- Protected API routes that require valid tokens
- User-specific filtering of database records
- Route guarding on the frontend for authenticated pages

This ensures that one user cannot access another user's data and that sensitive credentials are not stored in plain text.

7. Scheduling Logic Overview

One of the technically important features of the project is the smart study planning mechanism. The planner module reads pending task data, checks deadlines, estimated study hours, and priority levels, and then attempts to create a reasonable schedule across available dates.

The planner logic performs operations such as:

- Identifying pending tasks
- Sorting tasks according to urgency and priority
- Checking existing study sessions to avoid unnecessary overlap
- Assigning available hours within a daily study limit
- Creating study session records for future review and completion tracking

Although the current scheduling algorithm is rule-based and relatively simple, it provides meaningful automation and demonstrates how backend logic can convert raw task data into actionable planning output.

8. Data Visualization Overview

The project also includes a visualization layer through dashboard and analytics pages. Rather than presenting only textual data, the system displays information through:

- Bar charts
- Pie charts
- Line charts

- Summary cards
- Progress indicators

This improves user understanding and makes the platform more interactive. Visual analytics are especially useful for identifying progress trends, pending work distribution, and study consistency.

9. Working Principle

The overall technical workflow of the project can be summarized as follows:

1. A user creates an account through the signup page.
2. The login process verifies credentials and returns a JWT token.
3. The frontend stores the token and uses it for authenticated API requests.
4. The user creates courses and then adds tasks under those courses.
5. Task data is stored in PostgreSQL through backend APIs.
6. The planner module either manually stores or automatically generates study sessions.
7. Dashboard and analytics modules fetch aggregated data from backend endpoints.
8. The frontend displays the returned information in the form of lists, summaries, and charts.

10. Summary of Technical Design

From a technical standpoint, the project combines modern frontend development, API-based backend architecture, relational database design, secure authentication, and analytical visualization into one coherent system. The architecture is modular, practical, and scalable, making it suitable both as a functional student productivity application and as a strong academic software engineering project.

System Configuration

The system configuration of the " Learning Management System with Smart Study Planner" includes the hardware, software, development tools, and supporting libraries required to develop, run, and test the application successfully. Since the project is a full-stack web application, its configuration covers both frontend and backend environments along with database support.

The configuration has been chosen in such a way that the project can be developed and executed on a standard student or developer machine without requiring high-end infrastructure. At the same time, the selected tools are modern, reliable, and suitable for building a scalable academic web platform.

1. Hardware Requirements

The project does not demand specialized hardware, but a basic modern computer system is required for smooth development and execution.

- **Processor:** Intel Core i3 / AMD equivalent or above
- **RAM:** Minimum 4 GB, recommended 8 GB or more
- **Storage:** At least 1 GB free disk space for project files, dependencies, and database support
- **Display:** Standard laptop or desktop display
- **Internet Connection:** Required for dependency installation, documentation access, and updates

These hardware requirements are sufficient because the system is not computationally heavy. Most of the project functionality is related to web interface rendering, API communication, and structured database operations.

2. Software Requirements

The software environment is an important part of the system configuration because the project depends on multiple technologies working together.

- **Operating System:** Windows 10/11, Linux, or macOS
- **Python:** Version 3.9 or above
- **Node.js:** Version 18 or above
- **PostgreSQL:** Version 14 or above
- **Web Browser:** Google Chrome, Microsoft Edge, or Mozilla Firefox

The backend requires Python because FastAPI and related libraries run in the Python environment. The frontend requires Node.js because React, Vite, and associated packages depend on the Node ecosystem. PostgreSQL is used for relational data storage, while a modern browser is necessary to access and test the web interface.

3. Development Environment

For project development, testing, and debugging, the following tools are used:

- **Visual Studio Code** as the main code editor
- **Git** for version tracking and project management
- **Postman** or browser-based API tools for API testing
- **Terminal / Command Prompt / PowerShell** for running backend and frontend servers

The development environment supports efficient coding, package installation, debugging, API inspection, and build execution. It also allows the project to be maintained and updated easily.

4. Frontend Configuration

The frontend part of the application is configured with the following technologies:

- **React** for building the user interface
- **Vite** for development server support and frontend build process
- **Tailwind CSS** for styling and responsive design
- **Axios** for sending API requests
- **React Router** for navigation between pages
- **Recharts** for charts and graphical analytics

This frontend configuration ensures that the application remains interactive, responsive, and easy to navigate. It also supports fast page rendering and clean modular UI development.

5. Backend Configuration

The backend is configured using a modern Python-based API stack:

- **FastAPI** for backend routing and REST API creation
- **SQLAlchemy** for ORM-based database management
- **Pydantic** for data validation and schema enforcement
- **python-jose** for JWT token generation and verification
- **bcrypt** for password hashing and authentication security

The backend configuration allows the system to handle secure authentication, business logic, CRUD operations, planner generation, and analytics processing in a structured way.

6. Database Configuration

The application uses **PostgreSQL** as the database management system. The database stores all persistent records including:

- User information
- Course details
- Task records
- Study session schedules

- Progress and analytics-related data

Database connectivity is handled through SQLAlchemy configuration, and the database schema is defined through Python model classes. This setup ensures data consistency, relational integrity, and easier backend integration.

7. Execution Configuration

To run the system properly, the project requires separate execution of frontend and backend services:

1. The PostgreSQL database service must be active.
2. The FastAPI backend server must run on its configured port.
3. The React frontend development server must run separately.
4. The frontend communicates with the backend through API endpoints.

This execution configuration reflects the client-server design of the application and supports independent development, testing, and debugging of both layers.

Project Explanation

This project is a student-oriented web application that combines planning, management, and analysis into one academic platform.

1. User Authentication

The user first creates an account through the signup page. After successful registration, the user can log in using email and password. The backend validates credentials, generates a JWT token, and returns it to the frontend. Protected routes such as dashboard, courses, tasks, planner, and analytics are accessible only after authentication.

2. Course Management

After login, the user can create courses by entering the course title, description, and category. Courses act as containers for tasks. Users can also update and delete courses. When a course is deleted, related tasks and progress records are also cleaned up by backend logic.

3. Task Management

For each course, the user can create multiple tasks such as assignments, study topics, revision work, or project items. Each task stores deadline, estimated hours, priority, and current status. The user can filter tasks by status or course and mark them as completed when done.

4 .Smart Study Planner

The study planner is the main differentiating feature of the project. It supports two approaches:

- **Manual scheduling:** The user selects a task, date, and time slot to create a study session.
- **Automatic scheduling:** The backend planner algorithm fetches pending tasks, sorts them by deadline and priority, checks existing occupied slots, and assigns available study hours day by day.

The current planner assumes a maximum study limit per day and schedules sessions in a simple deterministic way. This makes the system useful while keeping the logic understandable and maintainable.

5. Dashboard and Analytics

The dashboard provides immediate visibility into the learner's academic state. It shows:

- Total number of courses
- Total tasks
- Completed tasks
- Study hours
- Course-wise progress
- Upcoming deadlines

The analytics page extends this with charts for weekly study hours, course completion percentages, and task distribution.

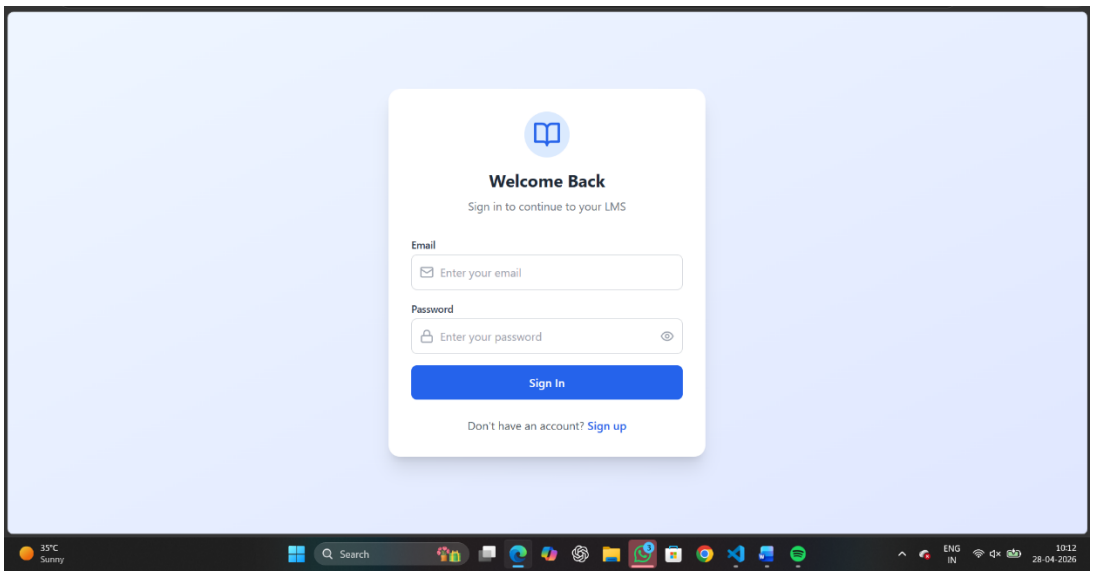
6. Data Flow

The frontend communicates with the backend using Axios. The backend processes each request using route handlers and database sessions. Responses are validated through schemas and rendered back in the UI. Authentication tokens are attached automatically through request interceptors.

Output screens

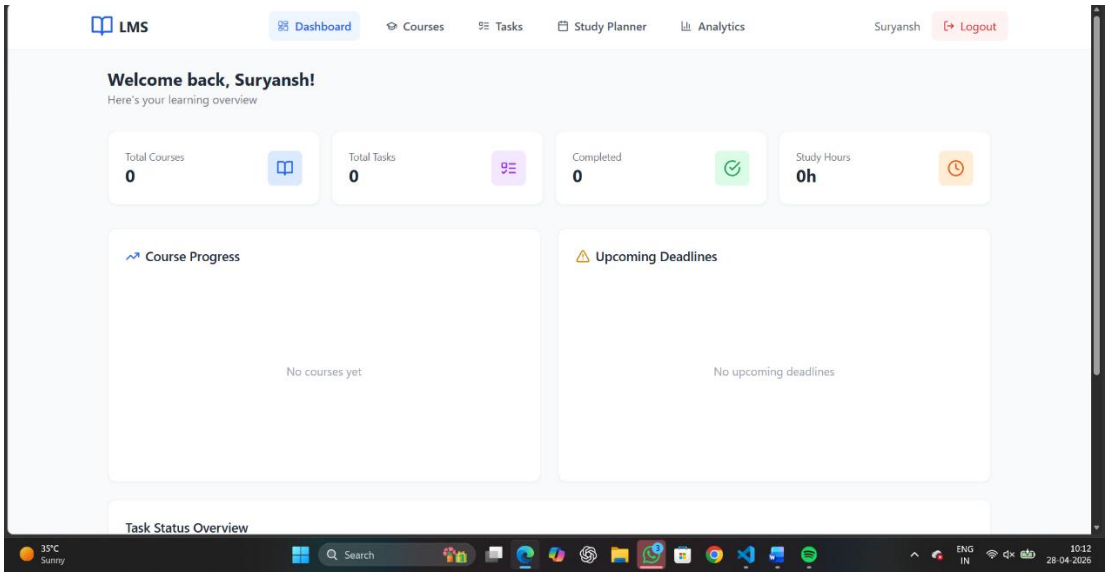
The following screenshots show the main interfaces of the developed Learning Management System with Smart Study Planner. These screens demonstrate the practical working of authentication, course and task management, study planning, and analytics modules.

1. Login Page



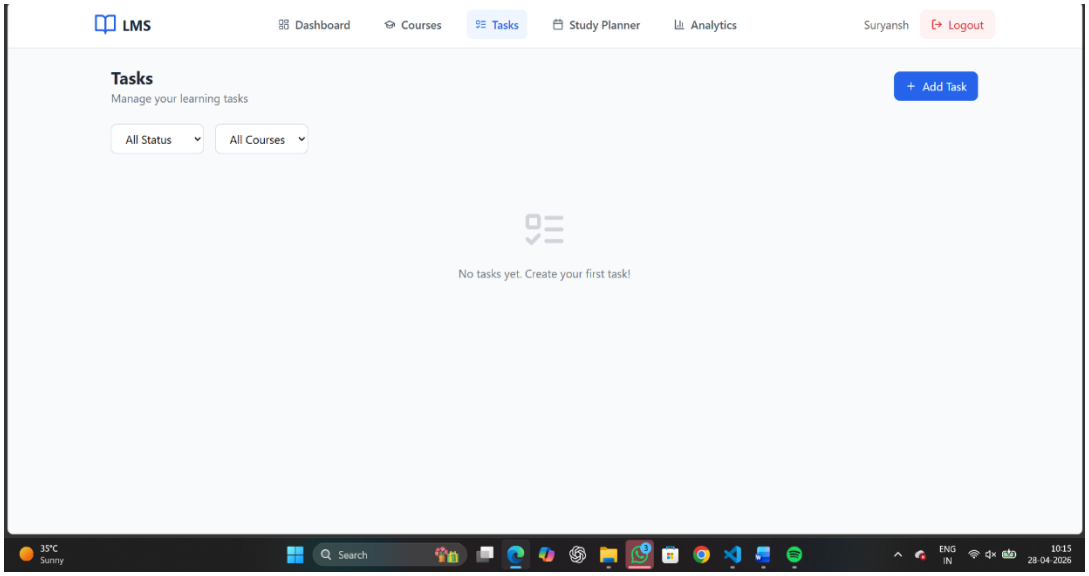
This screen allows the user to enter valid credentials and securely access the system.

2. Dashboard



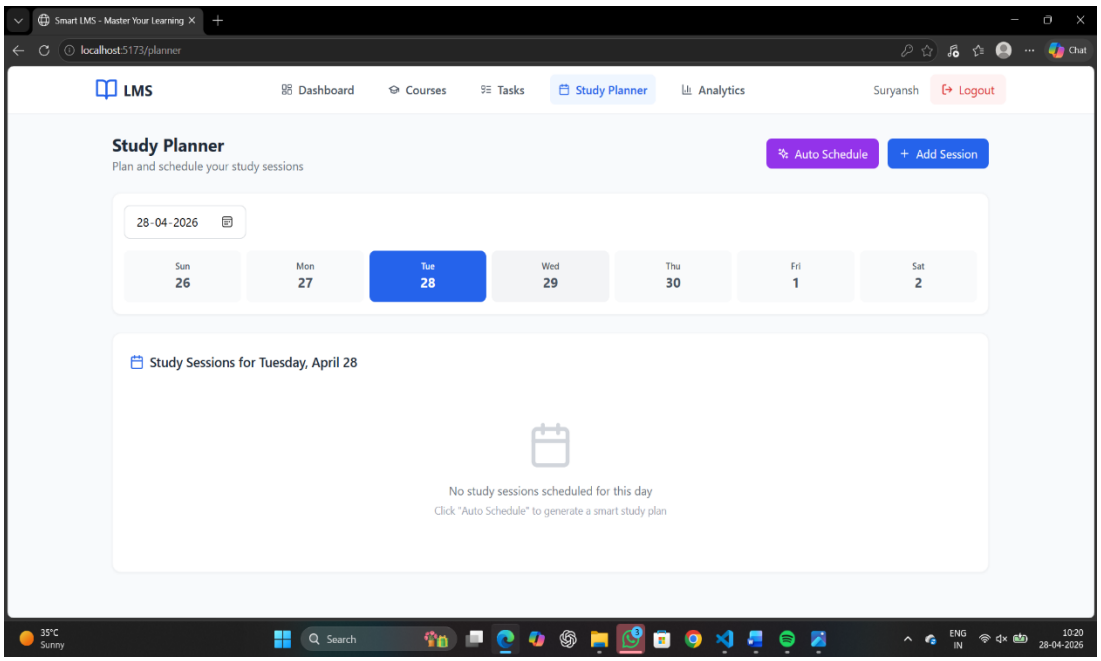
This screen displays summary information such as total courses, pending tasks, completed tasks, and progress overview.

3. Task Management



This screen shows task creation, task status handling, deadlines, and course-based organization.

4. Study Planner / Analytics



Result and Analysis

The developed system successfully integrates user authentication, course management, task management, study planning, and analytics into one web application. The final result is a usable academic productivity platform where a student can register, log in, create courses, add tasks, generate study schedules, and monitor progress visually.

Observed Outcomes

- The system successfully supports secure user registration and login, ensuring that each student's data remains private and accessible only after authentication.
- Users are able to create, update, view, and delete course records without difficulty, which confirms that the course management workflow functions correctly.
- Academic tasks can be added under specific courses with useful attributes such as deadline, priority, and estimated hours, making task records more meaningful and practical.
- The task completion feature works effectively and immediately updates the user workflow by reflecting whether work is pending or completed.
- The smart study planner is able to generate study sessions from pending tasks, showing that the application does more than simple data storage and provides actionable planning support.
- Dashboard summaries dynamically respond to user activity and provide a quick overview of total courses, pending tasks, completed tasks, and study workload.
- Analytics charts help present progress in a visual form, making it easier for users to understand their academic position at a glance.
- The frontend and backend communicate properly through API integration, and protected routes ensure that unauthorized users cannot access private academic data.

In practical use, the overall result of the project is that a student can manage the complete academic planning cycle within one platform. Instead of separately maintaining course lists, assignment reminders, and manual study timetables, the user can handle all these activities through a single system. This improves convenience, reduces fragmentation of information, and supports a more disciplined study routine.

The final application also demonstrates that the combination of planning, tracking, and analytics is more useful than a traditional task-list application. The user is not only able to record academic work, but also to organize it, execute it, and review progress over time. This makes the project suitable as both a practical student tool and a meaningful full-stack academic project.

Analysis

The performance of the system can be analyzed from both a functional and a practical point of view. Functionally, the project meets its major objectives. Core modules such as authentication, course handling, task management, study planning, and analytics are integrated into a single workflow. This indicates that the architecture is coherent and that the selected technologies are suitable for the problem domain.

From a usability perspective, the project simplifies a student's academic management process. In many existing study habits, users must manually decide what to do next, where to record it, and how to track completion. In this project, those steps are connected logically. A course contains tasks, tasks feed the planner, and planner activity contributes to dashboard and analytics output. This connected flow improves usability because the student does not have to repeatedly switch between unrelated tools.

The smart study planner is one of the most significant outcomes of the system. Its importance lies in the fact that it transforms passive academic data into an active study schedule. By considering deadline, priority, and estimated hours, the planner helps convert work items into a realistic sequence of sessions. Even though the current scheduling logic is rule-based and comparatively simple, it still provides a meaningful improvement over unstructured manual planning.

The dashboard and analytics components also add strong practical value. In academic systems, data is often stored but not interpreted. Here, the system uses stored task and study information to produce charts and summaries that reveal progress patterns. This supports self-monitoring, which is essential for learners who need regular feedback on workload, pending tasks, and overall consistency. As a result, the project encourages better planning behavior rather than only offering record maintenance.

From a software engineering perspective, the project demonstrates the advantage of modular full-stack design. The backend follows a domain-based structure where authentication, courses, tasks, planner logic, and analytics can be maintained separately. This improves readability, debugging, and future extension. On the frontend side, the use of reusable API handling and component-driven pages helps maintain a clean interaction flow and supports responsive user experience.

The project can also be analyzed in terms of educational relevance. It addresses a real problem commonly faced by students: difficulty in organizing workload across multiple subjects and deadlines. The system provides a practical answer by combining organization, prioritization, and visibility. Therefore, its value is not only technical but also academic and behavioral, since it can help students improve routine, discipline, and awareness of deadlines.

At the same time, some limitations remain in the current version. The system is mainly focused on individual student use and does not yet include teacher roles, classroom interaction, notifications, file sharing, or advanced recommendation features. The planner algorithm can also be made more intelligent in future by considering breaks, subject difficulty, preferred study times, and adaptive rescheduling. These limitations do not reduce the usefulness of the present implementation, but they identify clear directions for future enhancement.

Overall, the analysis shows that the project successfully fulfills its intended purpose. It is technically stable, functionally meaningful, and educationally relevant. The current version provides a strong foundation for future development and demonstrates how full-stack software can be used effectively to solve a real-world academic productivity problem.

Software Testing

Software testing was carried out to check the correctness, stability, and usability of the project modules.

The purpose of testing in this project was to ensure that all major functional components work reliably under normal user interaction. Since the application includes both frontend and backend modules, testing was focused not only on user interface behavior but also on API communication, authentication flow, validation logic, and overall data consistency. Special attention was given to the modules that directly affect student workflow, such as login, task creation, planner generation, and dashboard updates.

1. Testing Approach

- Functional testing
- API testing
- UI interaction testing
- Validation testing
- Build verification

Functional testing was performed to verify whether each core feature behaves according to the expected output. This included checking registration, login, course operations, task management, planner execution, and analytics loading. Each module was tested by providing valid and invalid inputs and observing the system response.

API testing was carried out to confirm that the frontend communicates correctly with backend endpoints. Request and response handling, token-based authorization, protected route access, and data retrieval behavior were reviewed to ensure correct integration between both layers.

UI interaction testing focused on checking page navigation, form submission, button actions, and state updates visible to the user. This helped ensure that the application remains usable and responsive during common academic workflows.

Validation testing was used to confirm that incorrect or incomplete input does not break the system and is handled properly through messages or restricted actions. This is especially important for login, task fields, and protected operations.

Build verification was used to ensure that the frontend project can be compiled successfully for production deployment and that the application structure is stable enough for demonstration and future extension.

2. Test Cases

Test Case	Input / Action	Expected Result	Status
User Signup	Enter valid email, username, password	New account created successfully	Passed
User Login	Enter registered email and password	Token generated and dashboard opened	Passed
Invalid Login	Wrong email or password	Error message shown	Passed
Add Course	Enter valid course data	Course saved and displayed	Passed
Edit Course	Update title or category	Updated course shown	Passed
Delete Course	Delete selected course	Course removed successfully	Passed
Add Task	Create task with course, deadline, priority	Task added to task list	Passed
Complete Task	Click complete on pending task	Task marked completed	Passed
Generate Planner	Trigger auto schedule	Study sessions created for pending tasks	Passed
Analytics Load	Open analytics page	Charts and summary data displayed	Passed
Protected Routes	Open secured page without login	Redirect to login page	Passed
Frontend Build	Run production build	Project builds successfully	Passed

3. Verification Summary

The results of software testing show that the major modules of the system are functioning correctly and in coordination with one another. Authentication works as expected, course and task operations perform successfully, planner generation produces study sessions, and analytics views are loaded without breaking the user experience.

The frontend application was also successfully built for production, which confirms that the client-side code is structurally stable. Backend data flow, protected routes, and API integration were validated through repeated functional checks. Error handling for invalid login attempts and unauthorized access was also observed to work correctly.

Based on the testing performed, the project can be considered stable, usable, and suitable for academic demonstration. Although more advanced testing methods such as automated unit testing, load testing, and security penetration testing can be added in future, the current level of testing is sufficient to confirm that the present implementation meets its functional requirements.

Conclusion

The **Learning Management System with Smart Study Planner** successfully demonstrates how modern web technologies can be used to solve an important real-world problem faced by students: managing academic workload efficiently.

The project provides a secure, organized, and user-friendly environment for handling courses, tasks, study sessions, and progress analytics. Its most valuable contribution is the integration of a smart scheduling mechanism, which transforms ordinary task data into an actionable study plan. This improves productivity, supports better time management, and reduces the chances of missing deadlines.

From a technical perspective, the project shows effective use of React, FastAPI, PostgreSQL, SQLAlchemy, JWT authentication, and chart-based analytics. From a practical perspective, it offers a usable tool for individual students and a scalable base for future educational software.

The project also highlights the importance of combining data management with intelligent assistance. Rather than functioning only as a digital record-keeping system, the application actively supports decision-making by helping users understand task urgency, distribute study effort, and monitor progress in a meaningful way. This makes the system more relevant to real student needs and increases its practical usefulness in everyday academic life.

Another important outcome of this project is the learning value it provides from a software development perspective. It demonstrates the successful integration of frontend design, backend API development, database connectivity, authentication, planning logic, and visualization into one complete system. Therefore, the project is not only useful as an end-user application, but also as a strong example of applied full-stack development in an educational context.

The present version of the system already fulfills its core purpose effectively, while also leaving clear opportunities for further enhancement. Features such as teacher access, reminders, calendar synchronization, AI-based recommendations, and more advanced scheduling logic can be added in future versions to make the platform even more intelligent and comprehensive.

In conclusion, the project meets its objectives and serves as a meaningful example of full-stack software development applied to academic planning and learner productivity. It provides both practical value to students and technical value as a project study, while establishing a solid foundation for future growth.

References

1. FastAPI Documentation. https://fastapi.tiangolo.com/
2. React Documentation. https://react.dev/
3. Vite Documentation. https://vitejs.dev/
4. SQLAlchemy Documentation. https://www.sqlalchemy.org/
5. PostgreSQL Documentation. https://www.postgresql.org/docs/
6. Recharts Documentation. https://recharts.org/
7. Tailwind CSS Documentation. https://tailwindcss.com/
8. JSON Web Token Introduction. https://jwt.io/introduction